

Number Theory

Catalan number

$$C_n = \frac{1}{n+1} \binom{2n}{n}$$

The Catalan number C_n is the solution for:

- 1. Balanced Parentheses:** The number of correct bracket sequences consisting of n opening and n closing brackets.
- 2. Word Problems:** The number of valid sequences of balanced words in formal language theory.
- 3. Bracketing Problems:** The number of ways to fully parenthesize an expression with n pairs of operands and operators.
- 4. Binary Search Trees:** The number of distinct binary search trees that can be formed with n nodes.
- 5. Sorted Trees:** The number of different ways to insert n elements into an empty binary search tree.
- 6. Full Binary Trees:** The number of full binary trees with $n+1$ leaves.
- 7. Triangulations of a Polygon:** The number of ways to triangulate a convex polygon with $n+2$ sides (i.e., the number of ways to partition a polygon into disjoint triangles using diagonals).
- 8. Non-Crossing Handshakes:** The number of ways n pairs of people can shake hands without any handshakes crossing.
- 9. Dyck Paths:** The number of paths along the edges of a grid that do not pass above the diagonal.
- 10. Stack Permutations:** The number of valid permutations of a sequence from 1 to n where each prefix is a valid stack permutation.
- 11. Valid Sequences:** The number of valid operations that can be performed on a stack to produce a given permutation of elements.
- 12. Mountain Ranges:** The number of ways to draw mountain ranges with n upstrokes and n downstrokes that never go below the starting point.

13. Schroeder's Second Problem: The number of ways to insert n pairs of parentheses into a sequence such that the resulting expression is correctly matched.

14. Planar Graphs: The number of certain types of planar graphs, such as non-crossing partitions of points in a plane.

15. Matrix Chain Multiplication: The number of ways to fully parenthesize a product of $n+1$ matrices.

16. Catalan Paths: The number of paths along the edges of a grid that start at the origin and do not pass above the line $y = x$.

Counting

Pascal

```
// C[n][r] = nCr
void buildPascal() {
    for (int i = 0; i <= N; i++) {
        for (int j = 0; j <= i; j++) {
            if (j == 0)
                C[i][j] = 1;
            else
                C[i][j] = C[i - 1][j] + C[i - 1][j - 1];
        }
    }
}
```

nCr any mod , n to $1e18$

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;
template<typename T>
using orderedset = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
#define int long long
#define ll long long
#define ld long double
#define endl '\n'
#define wady \
```

```

ios_base::sync_with_stdio(0); \
cin.tie(0); \
cout.tie(0);

void files() {
#ifdef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
}

ll fp(ll n, ll p, ll mod) {
    n = (n % mod + mod) % mod;
    ll ans = 1;
    while (p) {
        if (p & 1) ans = ans * n % mod;
        n = n * n % mod;
        p >>= 1;
    }
    return ans;
}

ll extended_euclid(ll a, ll b, ll &x, ll &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll d = extended_euclid(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

ll inverse(ll a, ll m) {
    ll x, y;
    ll g = extended_euclid(a, m, x, y);
    if (g != 1) return -1;
    return (x % m + m) % m;
}

// returns n! % mod without taking all the multiple factors of p into account
// that appear in the factorial
// mod = multiple of p
// O(mod) * log(n)

```

```

ll get_fact_mod(ll n, ll p, ll mod) {
    ll f[mod + 1];
    f[0] = 1;
    for (int i = 1; i <= mod; i++) {
        if (i % p) f[i] = f[i - 1] * i % mod;
        else f[i] = f[i - 1];
    }
    ll ans = 1;
    while (n > 1) {
        ans = ans * f[n % mod] % mod;
        ans = ans * fp(f[mod], n / mod, mod) % mod;
        n /= p;
    }
    return ans;
}

ll multiplicity(ll n, ll p) {
    ll ans = 0;
    while (n) {
        n /= p;
        ans += n;
    }
    return ans;
}

// nCr mod p^k
// O(p^k log n)
ll nCr(ll n, ll r, ll p, ll k) {
    if (n < r || r < 0 || n < 0) return 0;
    ll mod = 1;
    for (int i = 0; i < k; ++i)
        mod *= p;
    ll t = multiplicity(n, p) - multiplicity(r, p) - multiplicity(n - r, p);
    if (t >= k) return 0;
    ll ans = get_fact_mod(n, p, mod) * inverse(get_fact_mod(r, p, mod), mod) %
mod;
    ans = ans * inverse(get_fact_mod(n - r, p, mod), mod) % mod;
    ans = ans * fp(p, t, mod) % mod;
    return ans;
}

pair<ll, ll> CRT(ll a1, ll m1, ll a2, ll m2) {
    ll g = __gcd(m1, m2);
    if ((a2 - a1) % g != 0) return {-1, -1}; // No solution
    ll p, q;
    extended_euclid(m1 / g, m2 / g, p, q);

```

```

    ll mod = m1 / g * m2;
    ll x = a1 + (__int128) m1 * ((a2 - a1) / g % (m2 / g) * p % (m2 / g)) %
mod;
    x = (x % mod + mod) % mod;
    return {x, mod};
}

const int N = 1e6 + 5; // max mod
int spf[N];

void pre() {
    iota(spf, spf + N, 0);
    for (int i = 2; i * i < N; ++i) {
        if (spf[i] != i) continue;
        for (int j = i * i; j < N; j += i)
            spf[j] = min(spf[j], i);
    }
}

// O(mod logn logmod)
ll nCr(ll n, ll r, ll mod) {
    if (n < r || r < 0 || n < 0) return 0;
    pair<ll, ll> res = {0, 1};
    while (mod > 1) {
        int p = spf[mod], k = 0, cur = 1;
        while (mod % p == 0) {
            k++, cur *= p, mod /= p;
        }
        res = CRT(res.first, res.second, nCr(n, r, p, k), cur);
    }
    return res.first;
}

void solve(int tc) {
    int n, k, m;
    cin >> n >> k >> m;
    int boxes = (n + k - 1) / k;
    int rem = k;
    if (n % k) rem -= n % k;
    else rem -= k;
    cout << boxes << " " << nCr(rem + boxes - 1, boxes - 1, m) << endl;;
}

signed main() {
    wady

```

```

files();
int t = 1;
pre();
cin >> t;
int tc = 1;
while (t--)
    solve(tc++);
}

```

Stars and Bars

Number of non-negative integer solutions to

$$x_1 + x_2 + \dots + x_k = n \quad x_1, x_2, \dots, x_k \geq 0$$

$$\binom{n+k-1}{k-1}$$

$$\sum_{k=r}^n \binom{k}{r} = \binom{n+1}{r+1}$$

$$\sum_{k=0}^n k \binom{n}{k} = n \cdot 2^{n-1}$$

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}$$

$$\sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad n \geq 1$$

$$\sum_{k=0}^n \binom{n}{k} = 2^n$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=1}^n k^3 = \left(\frac{n(n+1)}{2} \right)^2$$

LDE and CRT

Diophantine Equations

A **Diophantine equation** is an equation where solutions are restricted to integers. They either have no solutions or infinitely many.

- **Linear Diophantine Equation:** Takes the form $ax+by=c$. $ax+by=c$
 - **Existence of Solutions:** A solution exists if and only if c is divisible by $\gcd(a,b)$.

- **Solving:** If a solution exists, one particular solution (x_0, y_0) can be found using the extended Euclidean algorithm (specifically for $ax+by=\gcd(a,b)$ and then scaling).
- **All Solutions:** Once a particular solution (x_0, y_0) for $ax+by=c$ is found, all integer solutions can be expressed as: $x=x_0-\gcd(a,b)br$ $y=y_0+\gcd(a,b)ar$ where r is any integer.

Congruence

Two integers a and b are **congruent modulo n** , written as $a \equiv b \pmod{n}$, if they have the same remainder when divided by n . This also means that $(a-b)$ is a multiple of n .

- **Properties:**
 - If $ax \equiv ay \pmod{n}$ and $\gcd(a,n)=d$, then $x \equiv y \pmod{n/d}$.
 - For large powers (e.g., $a^E \pmod{n}$), the exponent E can often be reduced using modular arithmetic properties like Fermat's Little Theorem ($a^{p-1} \equiv 1 \pmod{p}$ for prime p).

Linear Modular Equation

A linear modular equation is of the form $ax \equiv b \pmod{m}$.

- **Relationship to Diophantine Equations:** This equation is equivalent to a linear Diophantine equation: $ax-b=my$, which can be rewritten as $ax-my=b$.
- **Solutions:** Similar to linear Diophantine equations, solutions exist if b is divisible by $\gcd(a,m)$.

```
ll extended_euclid(ll a, ll b, ll &x, ll &y) {
    ll xx = y = 0;
    ll yy = x = 1;
    while (b) {
        ll q = a / b;
        ll t = b; b = a % b; a = t;
        t = xx; xx = x - q * xx; x = t;
        t = yy; yy = y - q * yy; y = t;
    }
    return a;
}

// a*x+b*y=c. returns valid x and y if possible.
// all solutions are of the form (x0 + k * b / g, y0 - k * b / g)
bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g) {
    if (a == 0 and b == 0) {
        if (c) return false;
        x0 = y0 = g = 0;
        return true;
    }
```

```

}
g = extended_euclid (abs(a), abs(b), x0, y0);
if (c % g != 0) return false;
x0 *= c / g;
y0 *= c / g;
if (a < 0) x0 *= -1;
if (b < 0) y0 *= -1;
return true;
}

void shift_solution(ll &x, ll &y, ll a, ll b, ll cnt) {
    x += cnt * b; // x = x + (b/g)
    y -= cnt * a; // y = y + (a/g)
}

// returns the number of solutions where x is in the range[minx, maxx] and y
// is in the range[miny, maxy]
ll find_all_solutions(ll a, ll b, ll c, ll minx, ll maxx, ll miny, ll maxy) {
    ll x, y, g;
    if (find_any_solution(a, b, c, x, y, g) == 0) return 0;
    if (a == 0 and b == 0) {
        assert(c == 0);
        return 1LL * (maxx - minx + 1) * (maxy - miny + 1);
    }
    if (a == 0) {
        return (maxx - minx + 1) * (miny <= c / b and c / b <= maxy);
    }
    if (b == 0) {
        return (maxy - miny + 1) * (minx <= c / a and c / a <= maxx);
    }
    a /= g, b /= g;
    ll sign_a = a > 0 ? +1 : -1;
    ll sign_b = b > 0 ? +1 : -1;
    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx) shift_solution(x, y, a, b, sign_b);
    if (x > maxx) return 0;
    ll lx1 = x;
    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx) shift_solution(x, y, a, b, -sign_b);
    ll rx1 = x;
    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny) shift_solution(x, y, a, b, -sign_a);
    if (y > maxy) return 0;
    ll lx2 = x;
    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy) shift_solution(x, y, a, b, sign_a);
    ll rx2 = x;
    if (lx2 > rx2) swap (lx2, rx2);

```

```

ll lx = max(lx1, lx2);
ll rx = min(rx1, rx2);
if (lx > rx) return 0;
return (rx - lx) / abs(b) + 1;
}

```

LDE with N variables

```

ll extended_euclid(ll a, ll b, ll &x, ll &y) {
    ll xx = y = 0;
    ll yy = x = 1;
    while (b) {
        ll q = a / b;
        ll t = b; b = a % b; a = t;
        t = xx; xx = x - q * xx; x = t;
        t = yy; yy = y - q * yy; y = t;
    }
    return a;
}

// a * x + b * y = c. returns valid x and y if possible.
bool find_any_solution(ll a, ll b, ll c, ll &x0, ll &y0, ll &g) {
    if (a == 0 and b == 0) {
        if (c) return false;
        x0 = y0 = g = 0;
        return true;
    }
    g = extended_euclid(abs(a), abs(b), x0, y0);
    if (c % g != 0) return false;
    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 *= -1;
    if (b < 0) y0 *= -1;
    return true;
}

// sum(a[i] * x[i]) = c, returns any valid solution. returns empty vector in
// case of failure
// x[i] can be any integer
// Complexity: O(n log(MAX))
// Optimization: You can reduce the number of variables to O(log (MAX))
// instead of n
// by only considering those values which reduces suffix gcds
vector<ll> find_any_solution(vector<ll> a, ll c) {
    int n = a.size();
    vector<ll> x;

```

```

bool all_zero = true;
for (int i = 0; i < n; i++) {
    all_zero &= a[i] == 0;
}
if (all_zero) {
    if (c) return {};
    x.assign(n, 0);
    return x;
}
ll g = 0;
for (int i = 0; i < n; i++) {
    g = __gcd(g, a[i]);
}
if (c % g != 0) return {};
if (n == 1) {
    return {c / a[0]};
}
vector<ll> suf_gcd(n);
suf_gcd[n - 1] = a[n - 1];
for (int i = n - 2; i >= 0; i--) {
    suf_gcd[i] = __gcd(suf_gcd[i + 1], a[i]);
}
ll cur = c;
for (int i = 0; i + 1 < n; i++) {
    ll x0, y0, g;
    // solve for a[i] * x + suf_gcd[i + 1] * (y / suf_gcd[i + 1]) = cur
    bool ok = find_any_solution(a[i], suf_gcd[i + 1], cur, x0, y0, g);
    assert(ok);
    {
        // trying to minimize x0 in case x0 becomes big
        // it is needed for this problem, not needed in general
        ll shift = abs(suf_gcd[i + 1] / g);
        x0 = (x0 % shift + shift) % shift;
    }
    x.push_back(x0);

    // now solve for the next suffix
    cur -= a[i] * x0;
}
x.push_back(a[n - 1] == 0 ? 0 : cur / a[n - 1]);
return x;
}

```

CRT

```

int egcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    int x1, y1;
    int g = egcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Solves system:  $x \equiv a_1 \pmod{m_1}$  and  $x \equiv a_2 \pmod{m_2}$ 
// Returns {x, lcm(m1, m2)} if solution exists, else {-1, -1}
pair<int, int> merge(int a1, int m1, int a2, int m2) {
    int x, y;
    int g = egcd(m1, m2, x, y);
    if ((a2 - a1) % g != 0) return {-1, -1};
    int lcm = m1 / g * m2;
    int diff = ((a2 - a1) / g) % (m2 / g);
    int mult = x % (m2 / g);
    int ret = (a1 + m1 * ((diff * mult) % (m2 / g))) % lcm;
    if (ret < 0) ret += lcm;
    return {ret, lcm};
}

```

inverse any mod

```

ll inverse(ll a, ll m) {
    ll x, y;
    ll g = extended_euclid(a, m, x, y);
    if (g != 1) {
        return -1;
    }
    return (x % m + m) % m;
}

```

Miller rabin

```

MillerRabin (Check if a number is prime)  $O(\log n)$ 
using u128 = __uint128_t;

/// return (s * m) % mod

```

```

ll mult(ll s, ll m, ll mod) {
    if (!m) return 0;
    ll ret = mult(s, m / 2, mod);
    ret = (ret + ret) % mod;
    if (m & 1) ret = (ret + s) % mod;
    return ret;
}

ll fp(ll b, ll p, ll mod) {
    ll res = 1;
    b %= mod;
    while (p > 0) {
        if (p & 1) res = (u128)res * b % mod;
        b = (u128)b * b % mod;
        p >>= 1;
    }
    return res;
}

bool check(ll n, ll a, ll d, ll s) {
    ll x = fp(a, d, n);
    if (x == 1 || x == n - 1) return 0;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1) return 0;
    }
    return 1;
}

bool MillerRabin(ll n) {
    if (n < 2) return 0;
    int r = 0;
    ll d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }
    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
        if (n == a) return 1;
        if (check(n, a, d, r)) return 0;
    }
    return 1;
}

```

Mobuis

Möbius Function

$$\begin{aligned} \mu(n) &= 1 \quad \text{if } n = 1, \quad \mu(p^2) = 0 \text{ for any prime } p, \quad \mu(n) = (-1)^k \text{ if } n \text{ is a product of } k \text{ distinct primes} \\ \mu(n) &= -1, \mu(1) = 1, \mu(p) = -1 \text{ for any prime } p \end{aligned}$$

Key Identity

$$\sum_{d|n} \mu(d) = 1 \quad \text{if } n = 1, \quad \sum_{d|n} \mu(d) = 0 \text{ for } n > 1$$

Möbius Inversion

$$\begin{aligned} \mu(n) &= \sum_{d|n} \mu(d) \\ \mu(n) &= \sum_{d|n} \mu(d) \left(\frac{n}{d}\right) = \sum_{d|n} \mu\left(\frac{n}{d}\right) \mu(d) \end{aligned}$$

Euler Totient Application

$$\begin{aligned} \sum_{d|n} \mu(d) &= n \\ \mu(n) &= \sum_{d|n} \mu(d) \frac{n}{d} \end{aligned}$$

Cyclotomic Formula

$$\begin{aligned} x^n - 1 &= \prod_{d|n} \Phi_d(x) \\ \Phi_n(x) &= \prod_{d|n} (x^d - 1)^{\mu(n/d)} \end{aligned}$$

Dirichlet Series

$$\sum_{n=1}^{\infty} \frac{\mu(n)}{n^s} = \frac{1}{\zeta(s)}, \quad \text{Re}(s) > 1$$

Useful Identities

$$\begin{aligned} \sum_{d|n} \mu(d) &= 2^{\omega(n)} \\ \sum_{d|n} \mu(d) \mu\left(\frac{n}{d}\right) &= \frac{n}{\phi(n)} \end{aligned}$$

Average Order

$$\lim_{x \rightarrow \infty} \frac{1}{x} \sum_{n \leq x} \mu(n) = 0$$

codes

```
mobius[1] = -1;
for (int i = 1; i < VALMAX; i++) {
    if (mobius[i]) {
        mobius[i] = -mobius[i];
        for (int j = 2 * i; j < VALMAX; j += i) { mobius[j] += mobius[i]; }
    }
}
```

PHI

Euler's totient function, also known as **phi-function** $\phi(n)$, counts the number of integers between 1 and n inclusive, which are coprime to n . Two numbers are coprime if their greatest common divisor equals 1

- If p is a prime number, then $\phi(p) = 1$ for all $1 < p$. Therefore we have:

$$\phi(p) = p - 1$$

- If p is a prime number and $k \geq 1$, then there are exactly $p^k - p^{k-1}$ numbers between 1 and p^k that are divisible by p . Which gives us:

$$\phi(p^k) = p^k - p^{k-1}$$

- If a and b are relatively prime, then:

$$\phi(ab) = \phi(a) \cdot \phi(b)$$

Codes

phi in sqrt

```

int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}

```

Phi from 1 to n

```

void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    phi[0] = 0;
    phi[1] = 1;
    for (int i = 2; i <= n; i++)
        phi[i] = i - 1;

    for (int i = 2; i <= n; i++)
        for (int j = 2 * i; j <= n; j += i)
            phi[j] -= phi[i];
}

```

Segmented phi

```

const long long MAX_RANGE = 1e6 + 6;
vector<long long> primes;
long long phi[MAX_RANGE], rem[MAX_RANGE];

vector<int> linear_sieve(int n) {
    vector<bool> composite(n + 1, 0);
    vector<int> prime;

    // 0 and 1 are not composite (nor prime)
    composite[0] = composite[1] = 1;

    for(int i = 2; i <= n; i++) {
        if(!composite[i]) prime.push_back(i);
    }
}

```

```

        for(int j = 0; j < prime.size() && i * prime[j] <= n; j++) {
            composite[i * prime[j]] = true;
            if(i % prime[j] == 0) break;
        }
    }
    return prime;
}

// To get the value of phi(x) for L <= x <= R, use phi[x - L].
void segmented_phi(long long L, long long R) {
    for(long long i = L; i <= R; i++) {
        rem[i - L] = i;
        phi[i - L] = i;
    }

    for(long long i : primes) {
        for(long long j = max(i * i, (L + i - 1) / i * i); j <= R; j += i) {
            phi[j - L] -= phi[j - L] / i;
            while(rem[j - L] % i == 0) rem[j - L] /= i;
        }
    }

    for(long long i = 0; i < R - L + 1; i++) {
        if(rem[i] > 1) phi[i] -= phi[i] / rem[i];
    }
}

```

```

const int N = 1e6 + 5;
int phi[N], g[N], lcm_sum[N];
vector<int> divi[N];
// phi[i] = number of integers j that are coprime with i (j <= i)
// g[i] = sum of gcd(a, b) for all 1 ≤ a < b ≤ i
// lcm_sum[i] = sum of LCM(j, i) for 1 ≤ j ≤ i
void euler() {
    for (int i = 1; i < N; ++i)
        phi[i] = i;
    for (int i = 2; i < N; ++i) {
        if (phi[i] == i) {
            for (int j = i; j < N; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
    for (int i = 1; i < N; ++i) {
        for (int j = i; j < N; j += i) {

```

```

        divi[j].push_back(i);
    }
}
for (int d = 1; d < N; ++d) {
    for (int k = d << 1; k < N; k += d) {
        g[k] += d * phi[k / d];
    }
}
for (int i = 1; i < N; ++i)
    g[i] += g[i - 1];
for (int n = 1; n < N; ++n) {
    lcm_sum[n] = 0;
    for (int &d: divi[n]) {
        lcm_sum[n] += phi[d] * d;
    }
    lcm_sum[n] = n * (lcm_sum[n] + 1) / 2; // +1 to account for d = n
}
}

```

Rules

$$(n) = \prod_{i=1}^k (1 + p_i + p_i^2 + \dots + p_i^{p_i-1}) = \prod_{i=1}^k \frac{p_i^{p_i+1} - 1}{p_i - 1}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

$$\sum_{i=1}^n i = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{24}$$

$$\sum_{i=1}^n i = \frac{(n(n+1))^2(2n^2+2n-1)}{12}$$

$$\sum_{i=1}^n x^i = \frac{x^{n+1} - 1}{x - 1} - 1$$

Sieve and linear and segmented sieve

```

// sieve
bool primes[N];
void sieve() {
    // Time Complexity : O(Nlog(log(N))).
    int N = 1000;
    memset(primes, true, sizeof primes);
    primes[0] = primes[1] = false;
}

```

```

    for (int i = 2; i * i <= N; i++) {
        if (primes[i]) {
            for (int j = i * i; j < N; j += i) {
                primes[j] = false;
            }
        }
    }
}
//

// linear

const int N = 1e5 + 9;

int spf[N];
vector<int> primes;
void sieve() {
    ()
}

```

```

#include <bits/stdc++.h>
using namespace std;

#define ll long long

// Generate all primes up to limit using sieve of eratosthenes
vector<int> sieve(int limit) {
    vector<bool> is_prime(limit + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int p = 2; p * p <= limit; ++p) {
        if (is_prime[p]) {
            for (int i = p * p; i <= limit; i += p) {
                is_prime[i] = false;
            }
        }
    }
    vector<int> primes;
    for (int p = 2; p <= limit; ++p) {
        if (is_prime[p]) {
            primes.push_back(p);
        }
    }
    return primes;
}

```

```

// Generate all primes from l to r using segmented sieve in O((r - l) log (r)
+ sqrt(r))
vector<ll> segmented_sieve(ll l, ll r) {
    if (l == 1) {
        l++;
    }
    int limit = sqrtl(r);
    while ((ll) limit * limit <= r) limit++;
    while ((ll) limit * limit > r) limit--;
    auto primes = sieve(limit);
    vector<bool> is_prime(r - l + 1, true);
    for (ll p : primes) {
        ll start = max((ll)p * p, (ll)(l + p - 1) / p * p);
        for (ll j = start; j <= r; j += p) {
            is_prime[j - l] = false;
        }
    }
    vector<ll> vec;
    for (ll i = l; i <= r; ++i) {
        if (is_prime[i - l]) {
            vec.push_back(i);
        }
    }
    return vec;
}

int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int t;
    cin >> t;
    while (t--) {
        ll l, r;
        cin >> l >> r;
        auto primes = segmented_sieve(l, r);
        for (auto p: primes) {
            cout << p << '\n';
        }
    }
    return 0;
}

// https://www.spoj.com/problems/PRINT/

```

sieve to 1e9

0.5 sec

```
vector<int> sieve(const int N, const int Q = 17, const int L = 1 << 15) {
    static const int rs[] = {1, 7, 11, 13, 17, 19, 23, 29};
    struct P {
        P(int p) : p(p) {}
        int p; int pos[8];
    };
    auto approx_prime_count = [] (const int N) -> int {
        return N > 60184 ? N / (log(N) - 1.1)
            : max(1., N / (log(N) - 1.11)) + 1;
    };

    const int v = sqrt(N), vv = sqrt(v);
    vector<bool> isp(v + 1, true);
    for (int i = 2; i <= vv; ++i) if (isp[i]) {
        for (int j = i * i; j <= v; j += i) isp[j] = false;
    }

    const int rsize = approx_prime_count(N + 30);
    vector<int> primes = {2, 3, 5}; int psize = 3;
    primes.resize(rsize);

    vector<P> sprimes; size_t pbeg = 0;
    int prod = 1;
    for (int p = 7; p <= v; ++p) {
        if (!isp[p]) continue;
        if (p <= Q) prod *= p, ++pbeg, primes[psize++] = p;
        auto pp = P(p);
        for (int t = 0; t < 8; ++t) {
            int j = (p <= Q) ? p : p * p;
            while (j % 30 != rs[t]) j += p << 1;
            pp.pos[t] = j / 30;
        }
        sprimes.push_back(pp);
    }

    vector<unsigned char> pre(prod, 0xFF);
    for (size_t pi = 0; pi < pbeg; ++pi) {
        auto pp = sprimes[pi]; const int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            const unsigned char m = ~(1 << t);
            for (int i = pp.pos[t]; i < prod; i += p) pre[i] &= m;
        }
    }
}
```

```

const int block_size = (L + prod - 1) / prod * prod;
vector<unsigned char> block(block_size); unsigned char* pblock =
block.data();
const int M = (N + 29) / 30;

for (int beg = 0; beg < M; beg += block_size, pblock -= block_size) {
    int end = min(M, beg + block_size);
    for (int i = beg; i < end; i += prod) {
        copy(pre.begin(), pre.end(), pblock + i);
    }
    if (beg == 0) pblock[0] &= 0xFE;
    for (size_t pi = pbeg; pi < sprimes.size(); ++pi) {
        auto& pp = sprimes[pi];
        const int p = pp.p;
        for (int t = 0; t < 8; ++t) {
            int i = pp.pos[t]; const unsigned char m = ~(1 << t);
            for (; i < end; i += p) pblock[i] &= m;
            pp.pos[t] = i;
        }
    }
    for (int i = beg; i < end; ++i) {
        for (int m = pblock[i]; m > 0; m &= m - 1) {
            primes[psize++] = i * 30 + rs[__builtin_ctz(m)];
        }
    }
}
assert(psize <= rsize);
while (psize > 0 && primes[psize - 1] > N) --psize;
primes.resize(psize);
return primes;
}

```